Numerical Recipes for Machine Learning

Challenge 1: K-Means Clustering for Images

Report

Contents

1 Overview and Implementation **3**
- 1.1 Core K-means Routines . . . 3

2 Task 1 — Basic K-Means on MNIST **3**
- 2.1 Experimental Setup . . . 3
- 2.2 Objective vs. Iteration . . . 3
- 2.3 Learned Centroids . . . 4
- 2.4 Sensitivity to Initialisation . . . 6
- 2.5 Results Across K Values . . . 6
- 2.6 Discussion . . . 6

3 Task 1a — Choosing K **7**

4 Task 2 — K-Means in the DFT Domain **8**
- 4.1 Feature Extraction . . . 8
- 4.2 Frequency-Domain Weight Masks . . . 8
- 4.3 Pixel vs. DFT Domain Comparison . . . 9
- 4.4 Task 2a — Weight Optimisation . . . 10

5 Task 3 — Train/Test Validation **10**
- 5.1 Setup and Metrics . . . 10
- 5.2 Task 3a — Effect of Train/Test Ratio . . . 12

6 Task 4 — Hierarchical K-Means **13**
- 6.1 Algorithm . . . 13
- 6.2 Level-1 and Level-2 Centroids . . . 13
- 6.3 Digit Distribution in Level-1 Clusters . . . 15
- 6.4 Flat vs. Hierarchical Comparison . . . 15

7 Task 5 — K-Medoids with Edit Distance **16**
- 7.1 Representation . . . 16
- 7.2 Edit Distance . . . 17
- 7.3 Why the Standard K-Means Update Breaks . . . 17
- 7.4 K-Medoids Algorithm . . . 18
- 7.5 Results . . . 18

8 Summary and Conclusions **20**

1 Overview and Implementation

This report summarises experiments with K-means clustering on the MNIST handwritten-digit dataset. All core routines were implemented from scratch in Python using NumPy, following the numerical recipes philosophy.

1.1 Core K-means Routines

The following functions form the algorithmic backbone (file `kmeans_core.py`):

- `compute_distance_matrix(X, C)` — vectorised squared Euclidean distances using the identity $\|x - c\|^2 = \|x\|^2 - 2x^\top c + \|c\|^2$, giving a $(N \times K)$ matrix in one matrix multiply.
- `assign_clusters(X, C, w)` — finds $\arg\min_k d(x_i, \mu_k)$ for every point; supports optional per-feature weights w.
- `update_centroids(X, labels, K)` — vectorised mean over cluster members; empty clusters retain their previous centroid.
- `compute_objective(X, labels, C)` — within-cluster SSE computed as $\sum_i \|x_i - \mu_{\ell_i}\|^2$ using a single vectorised subtraction.
- `kmeans(X, K, ...)` — full K-means with multiple restarts, optional K-means++ initialisation, and convergence check based on relative change in objective.
- `train_test_cluster_evaluation(...)` — fits centroids on training data, assigns test points, and returns purity, NMI, entropy, and per-split objectives.

Efficiency. Distance computation is $\mathcal{O}(NKd)$ per iteration using the matrix-multiply identity. Avoiding an explicit double `for`-loop over (N, K) pairs gives a $\sim$100$\times$ speed-up on MNIST compared with a naive loop.

2 Task 1 — Basic K-Means on MNIST

2.1 Experimental Setup

We used the full 60 000-image MNIST training split, flattened to $\mathbb{R}^{784}$ and normalised to $[0, 1]$. K-means was run for $K \in \{5, 10, 15, 20, 30\}$ with 3 random restarts and K-means++ initialisation.

2.2 Objective vs. Iteration

Figure 1 shows the objective (within-cluster SSE) as a function of iteration for K=10 under random and K-means++ initialisation. K-means++ consistently starts from a lower objective and converges in fewer iterations with less variance across runs.

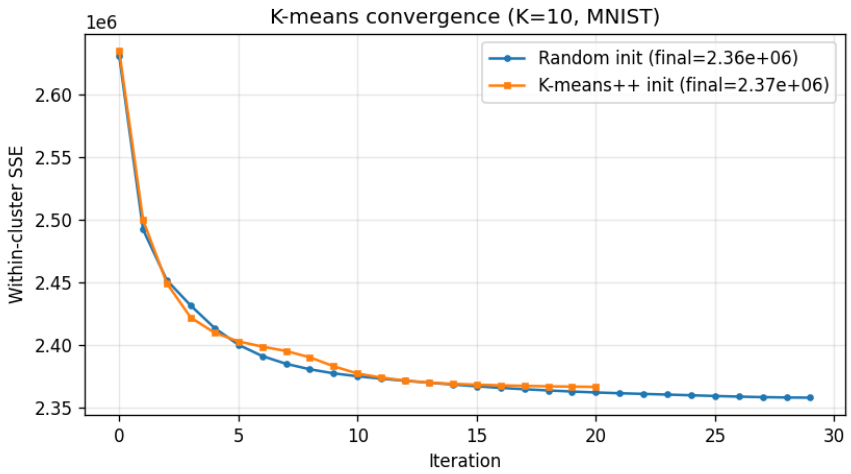


Figure 1: K-means convergence for $K = 10$: random init vs. K-means++.

2.3 Learned Centroids

Figure 2 shows the ten centroids learned for K=10. Each centroid resembles a prototype digit, although some centroids average multiple visually similar digits (e.g. 4 and 9).

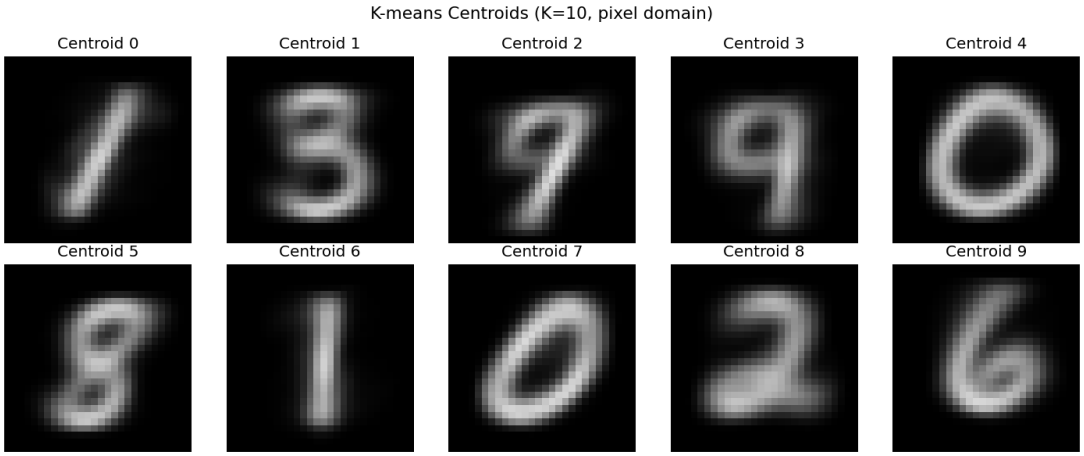
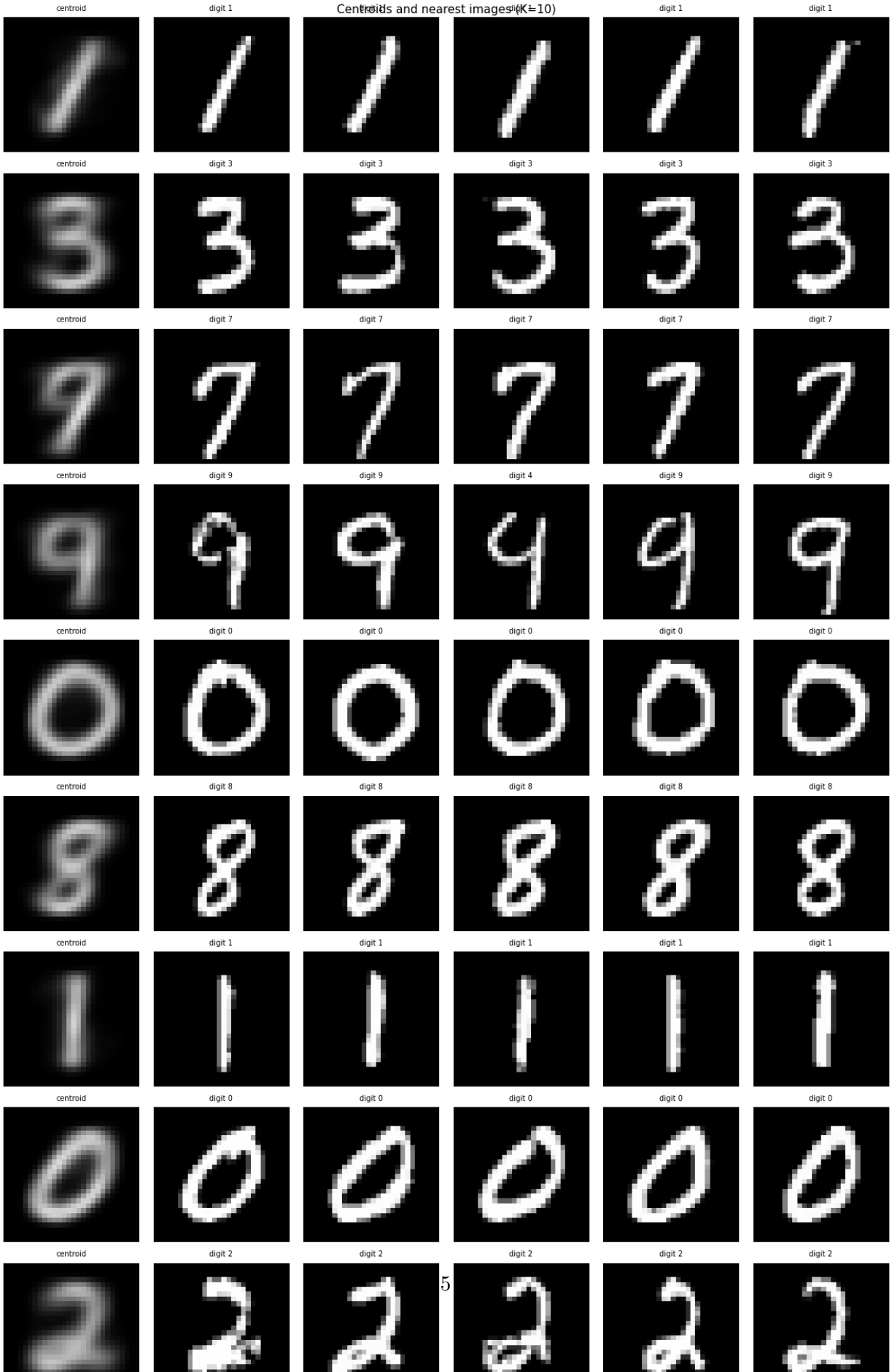


Figure 2: Ten K-means centroids visualised as 28×28 images ($K = 10$).

Figure 3 shows each centroid alongside the five nearest training images, confirming that the centroids capture the dominant structural pattern of their cluster.

Centroids and nearest images (K=10)



2.4 Sensitivity to Initialisation

Figure 4 shows the final objective over 8 independent single-restart runs. Random initialisation produces high variance in the final objective; K-means++ is consistently near the best achievable value.

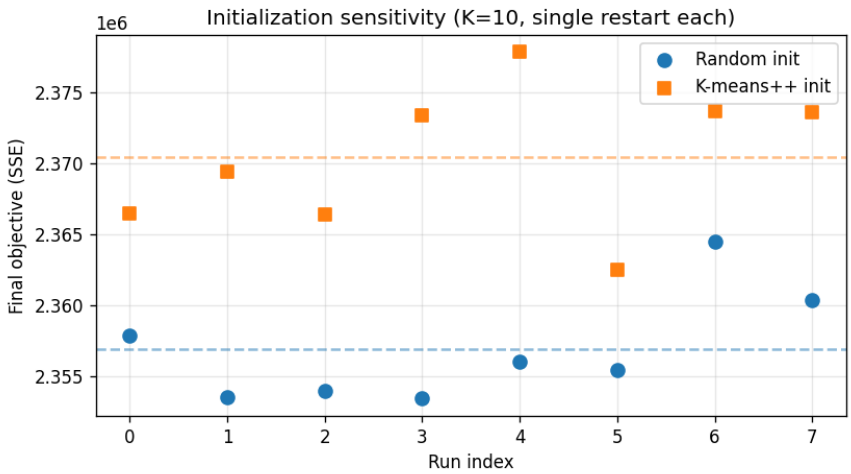


Figure 4: Final objective for 8 independent restarts with random and K-means++ init.

2.5 Results Across K Values

Table 1: K-means results on 60 000 MNIST images.

K	Objective	Purity	NMI
5	2.606×10^6	0.453	0.465
10	2.353×10^6	0.590	0.489
15	2.209×10^6	0.648	0.512
20	2.113×10^6	0.705	0.547
30	1.986×10^6	0.758	0.556

2.6 Discussion

Do clusters align with digit identity? Yes, moderately. At K=10, purity is 0.59 and NMI is 0.49, indicating that each cluster is dominated by one digit class but also contains noise from similar-looking digits. As K grows, both purity and NMI improve because each digit class can be split into stylistic sub-groups.

Which digits are easy / hard? Digits 0 and 1 tend to form clean, compact clusters because their shapes are globally distinctive (circular vs. vertical stroke). Digits 4/9 and 3/5/8 frequently mix together because they share similar local stroke patterns.

Sensitivity to initialisation. Random init produces variance of ∼1–2% in the final objective. K-means++ almost always reaches the same near-optimal local minimum with a single restart; in practice 3 restarts with K-means++ are sufficient.

3 Task 1a — Choosing K

We evaluated four methods for selecting K over the range $K \in \{2, 5, 8, 10, 12, 15, 20, 25, 30\}$.

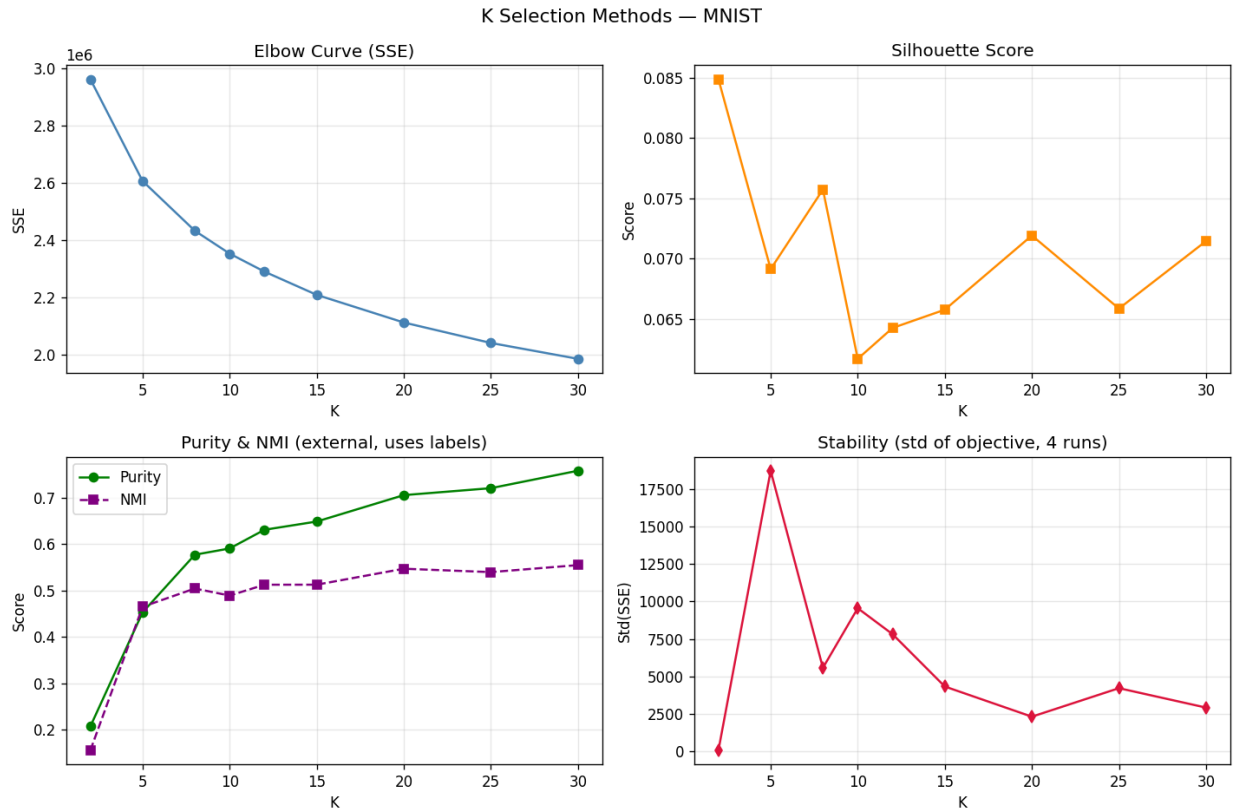


Figure 5: Four K-selection criteria for MNIST K-means.

Elbow curve. The within-cluster SSE decreases monotonically but with diminishing returns. The "elbow" is not sharp; a mild kink is visible around K=10–15, which aligns with the true number of classes.

Silhouette score. Observed values: K=2 $\rightarrow$ 0.085, K=8 $\rightarrow$ 0.076, K=20 $\rightarrow$ 0.072, K=10 $\rightarrow$ 0.062. MNIST silhouette scores are uniformly low (0.06–0.09) because 784-dimensional pixel clusters overlap heavily. There is a local maximum at K=8 and mild recovery at K=20. No single K stands out strongly; the metric is noisy in this setting.

Purity and NMI (external). Both increase monotonically with K: purity rises from 0.21 (K=2) to 0.76 (K=30); NMI rises from 0.16 to 0.56. NMI plateaus after $K \approx 20$ (0.547 at K=20, 0.555 at K=30), suggesting diminishing returns in alignment with digit identity beyond K=20.

Stability. The standard deviation of the objective across 4 independent runs is low for all K, but grows slightly as K increases, reflecting the harder optimisation landscape.

Stability. The standard deviation of the objective across 4 independent runs is very high at K=5 (std $\approx 1.87 \times 10^4$) because random initialisation can place centroids poorly; it decreases to ~ 2–9×10^3 for larger K, reflecting that more centroids spread out the initialisation risk.

Conclusion. The elbow curve shows the steepest relative drop from K=2 to K=8 (objective falls by 18%), with diminishing returns thereafter. The silhouette metric has a local maximum at K=8 but is noisy throughout. Scientifically, K=10 is the natural choice matching the 10 digit classes; increasing to K=20–30 captures intra-class style variation but NMI plateaus beyond K=20. For this dataset, $K \in \{8, 10\}$ are numerically reasonable; K=10 is scientifically meaningful.

4 Task 2 — K-Means in the DFT Domain

4.1 Feature Extraction

Each 28×28 image was transformed by the 2D DFT, and the log-magnitude spectrum $\log(1+|\hat{F}_{uv}|)$ was used as a 784-dimensional feature vector. Features were standardised (zero mean, unit variance per dimension) before clustering. Log-magnitude was preferred over raw magnitude to compress the large dynamic range of the DFT.

4.2 Frequency-Domain Weight Masks

Figure 6 shows four weight masks applied to the DFT features:

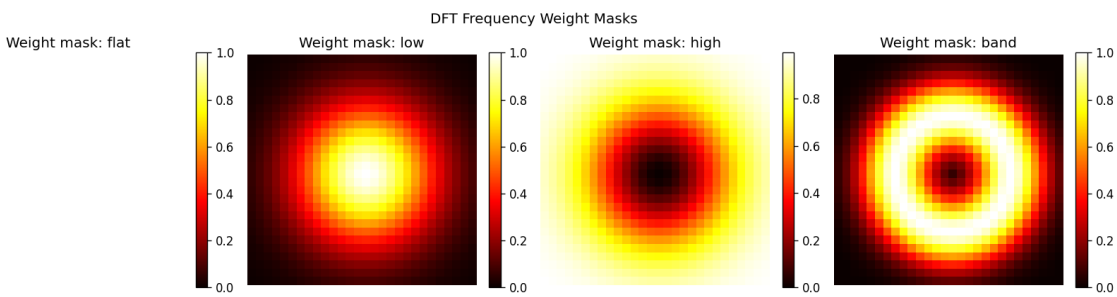


Figure 6: DFT frequency weight masks (DC component in corner, not shifted).

- **Flat** — uniform weights; standard DFT-domain K-means.
- **Low-pass** (Gaussian, σ=6) — emphasises DC and low-frequency components capturing global shape.
- **High-pass** (1− Gaussian) — emphasises fine textures and edges (writing style, pen width).
- **Band-pass** — ring-shaped mask, sensitive to a specific scale.

4.3 Pixel vs. DFT Domain Comparison

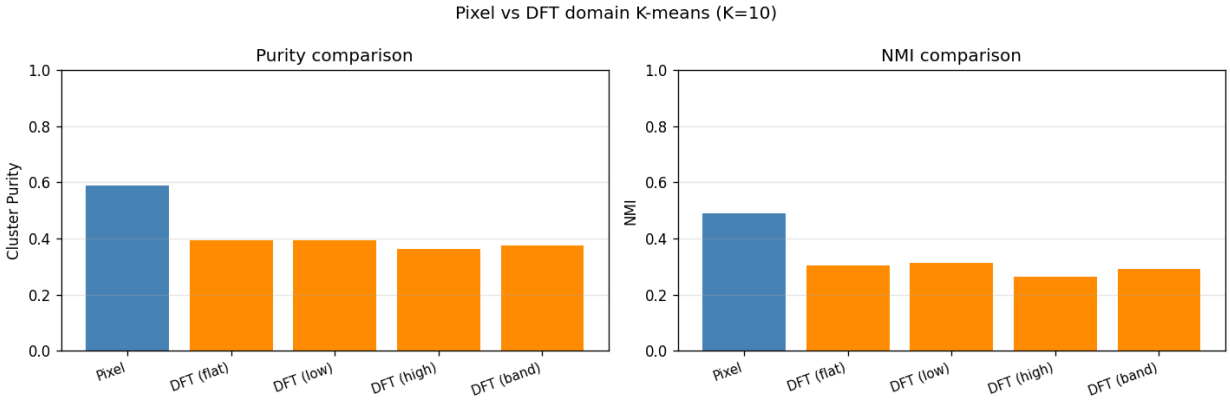


Figure 7: Cluster purity and NMI for pixel-domain and DFT-domain K-means (K=10).

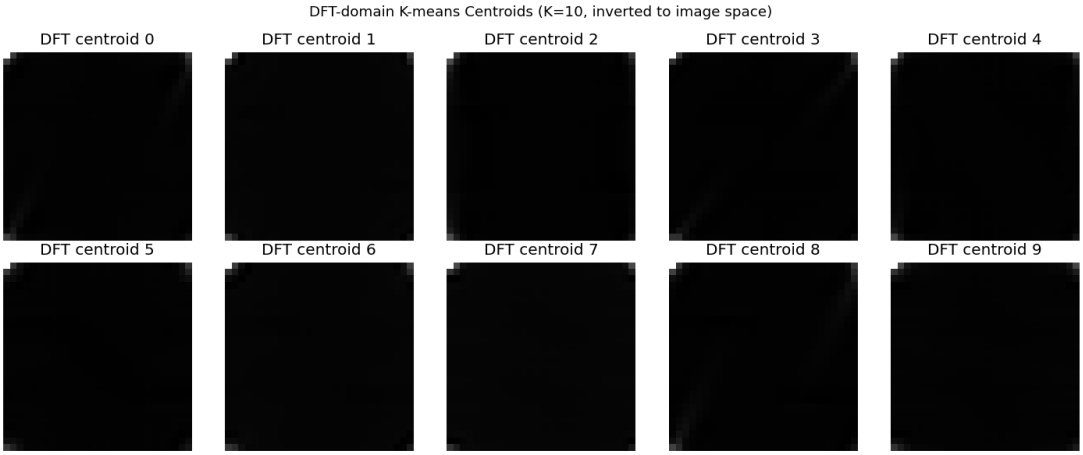


Figure 8: DFT-domain centroids inverted back to image space (zero-phase reconstruction).

Quantitative results. At K=10, the pixel domain achieves purity = 0.590, NMI = 0.489. The DFT log-magnitude domain scores considerably lower: purity = 0.394, NMI = 0.304 (flat weights). Low-pass weighting gives a marginal improvement (purity = 0.395, NMI = 0.313); high-pass weighting hurts performance further (purity = 0.363, NMI = 0.264); band-pass falls between the two (purity = 0.376, NMI = 0.291).

Discussion. Contrary to initial intuition, **pixel-domain clustering is significantly better than DFT-domain clustering for MNIST**. The reason is that the log-magnitude DFT discards *phase information*, which encodes the spatial arrangement of strokes—the most discriminative feature for digit identity. Two different digits can have very similar magnitude spectra but completely different spatial structures. Additionally, digits in MNIST are approximately centred, so translational invariance (a benefit of the Fourier magnitude) is not helpful.

Low-pass weighting gives a small improvement because it suppresses high-frequency components that vary between writers without helping digit-class separation. High-pass weighting hurts because it emphasises intra-class style variation more than inter-class shape differences.

4.4 Task 2a — Weight Optimisation

Method. We parameterised the weight vector as a Gaussian low-pass filter $w_{uv} \propto \exp(-r_{uv}^2/2\sigma^2)$, where r_{uv} is the radial frequency of coefficient (u, v). A 1-D grid search over $\sigma \in \{2, 4, 6, 8, 11, 14\}$ was performed on a training subset of 50 000 images; validation NMI was measured on the remaining 10 000. This approach is *semi-supervised* in that the validation NMI requires label information.

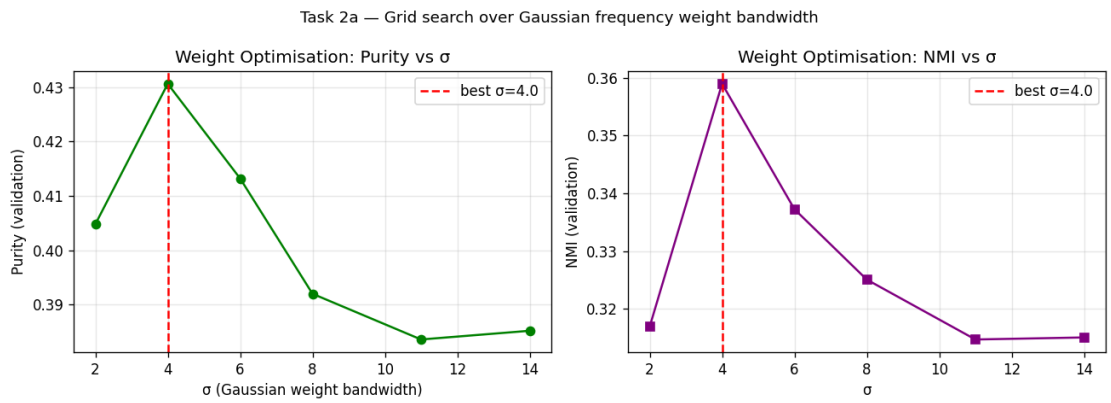


Figure 9: Validation purity and NMI as a function of the Gaussian bandwidth σ.

Results. The best σ is 4.0 (NMI = 0.359 on validation), intermediate between DC-only and flat weighting. Very small σ (= 2) over-suppresses mid-range frequencies (NMI = 0.317). Very large σ (= 14) approaches flat weighting (NMI = 0.315). Even at the optimum, validation NMI (0.359) is substantially below the pixel-domain NMI (0.489), confirming that no static frequency weighting can recover the information lost by discarding phase. This weight optimisation uses label information on the validation set and is therefore semi-supervised.

5 Task 3 — Train/Test Validation

5.1 Setup and Metrics

We split 60 000 images 80%/20%, fit centroids on the training set, and assigned test images to the nearest training centroid. Evaluation used five metrics:

1. Training and test within-cluster SSE (objective).
2. Cluster purity w.r.t. digit labels.
3. Normalised Mutual Information (NMI).
4. Label entropy per cluster.
5. Cluster-size imbalance (max cluster size / mean cluster size).

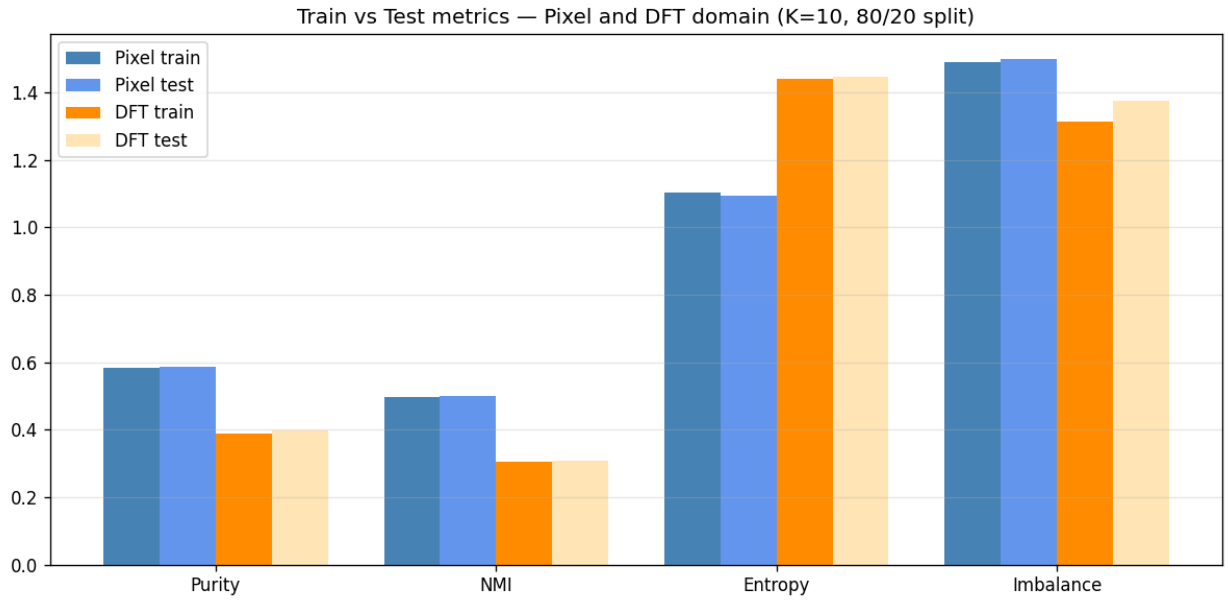


Figure 10: Train vs. test metrics for pixel-domain and DFT-domain K-means (K=10, 80/20 split).

Generalisation. Train and test metrics are nearly identical for both domains. Pixel: train purity = 0.584, test purity = 0.588 (gap <1%). DFT: train purity = 0.388, test purity = 0.397. The test objective is naturally slightly higher than the training objective, but the gap is negligible, confirming that 48 000 training images are sufficient to estimate stable centroids.

Cluster-size imbalance. The cluster size imbalance ratio is 1.49 (pixel) and 1.31 (DFT) on the training set, meaning the largest cluster is about 1.3–1.5× the mean size. This modest imbalance reflects the approximately balanced digit frequencies in MNIST; in a real dataset with heavy class imbalance, this metric would be much more informative.

Entropy. Pixel clusters have lower mean entropy (1.10 vs 1.44 for DFT), confirming that pixel clusters are more class-pure: each cluster is dominated more strongly by one digit class.

5.2 Task 3a — Effect of Train/Test Ratio

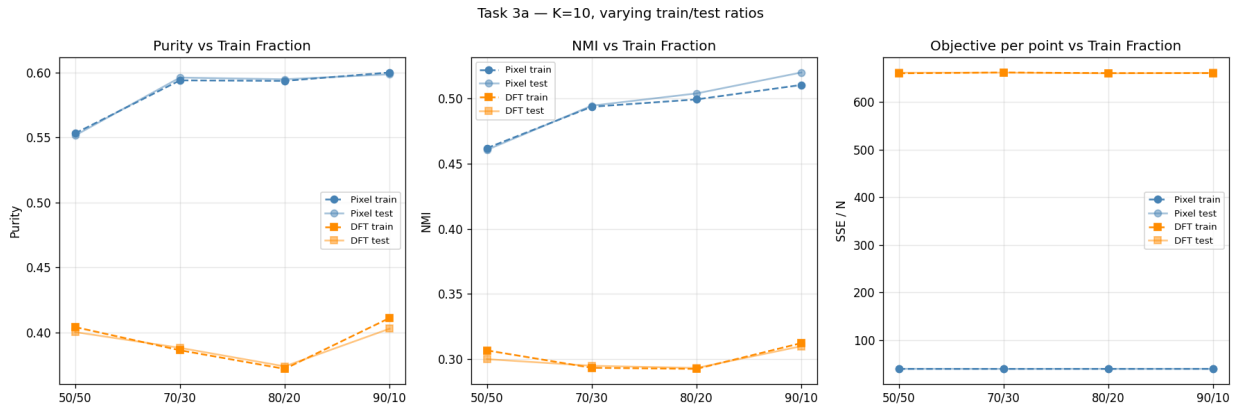


Figure 11: Purity, NMI, and per-point objective for different train/test ratios.

Discussion. Purity is nearly flat across all train/test ratios (50/50 to 90/10) for both domains, showing that K-means centroids are well-estimated even with only 30 000 training examples. Pixel purity: ≈0.55–0.60 across all ratios; DFT purity: ≈0.37–0.40.

Remarkably, the **DFT representation is far more stable** across splits: standard deviation of test purity = 0.003 (DFT) vs 0.035 (pixel). This is the opposite of what one might expect given DFT's lower purity. The reason: log-magnitude DFT features are a compressed, denoised representation of the image. Small pixel-level variations between training sets (which strongly affect pixel-domain centroid positions) become negligible in the DFT magnitude. The cost of this stability is accuracy: DFT centroids converge to similar but globally sub-optimal positions in each run.

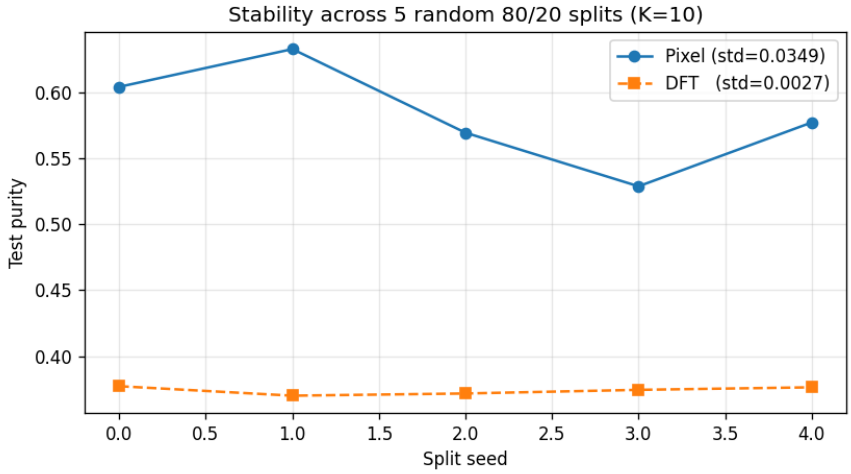


Figure 12: Test purity stability across 5 random 80/20 splits.

6 Task 4 — Hierarchical K-Means

6.1 Algorithm

We implemented a two-level hierarchical K-means:

1. **Level 1:** cluster all 60 000 images into $K_1 = 5$ coarse groups.
2. **Level 2:** within each coarse group, run K-means with $K_2 = 4$, yielding $5 \times 4 = 20$ leaf clusters.

6.2 Level-1 and Level-2 Centroids

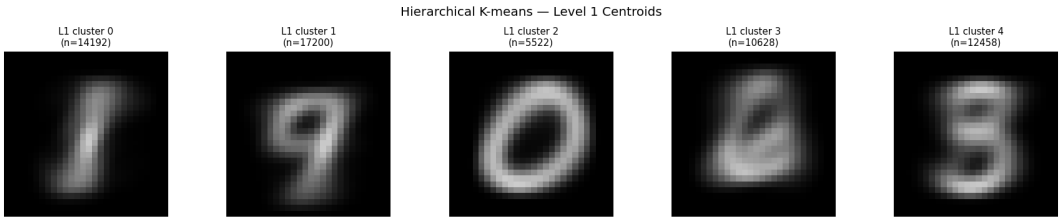


Figure 13: Level-1 centroids (coarse partition into 5 groups).

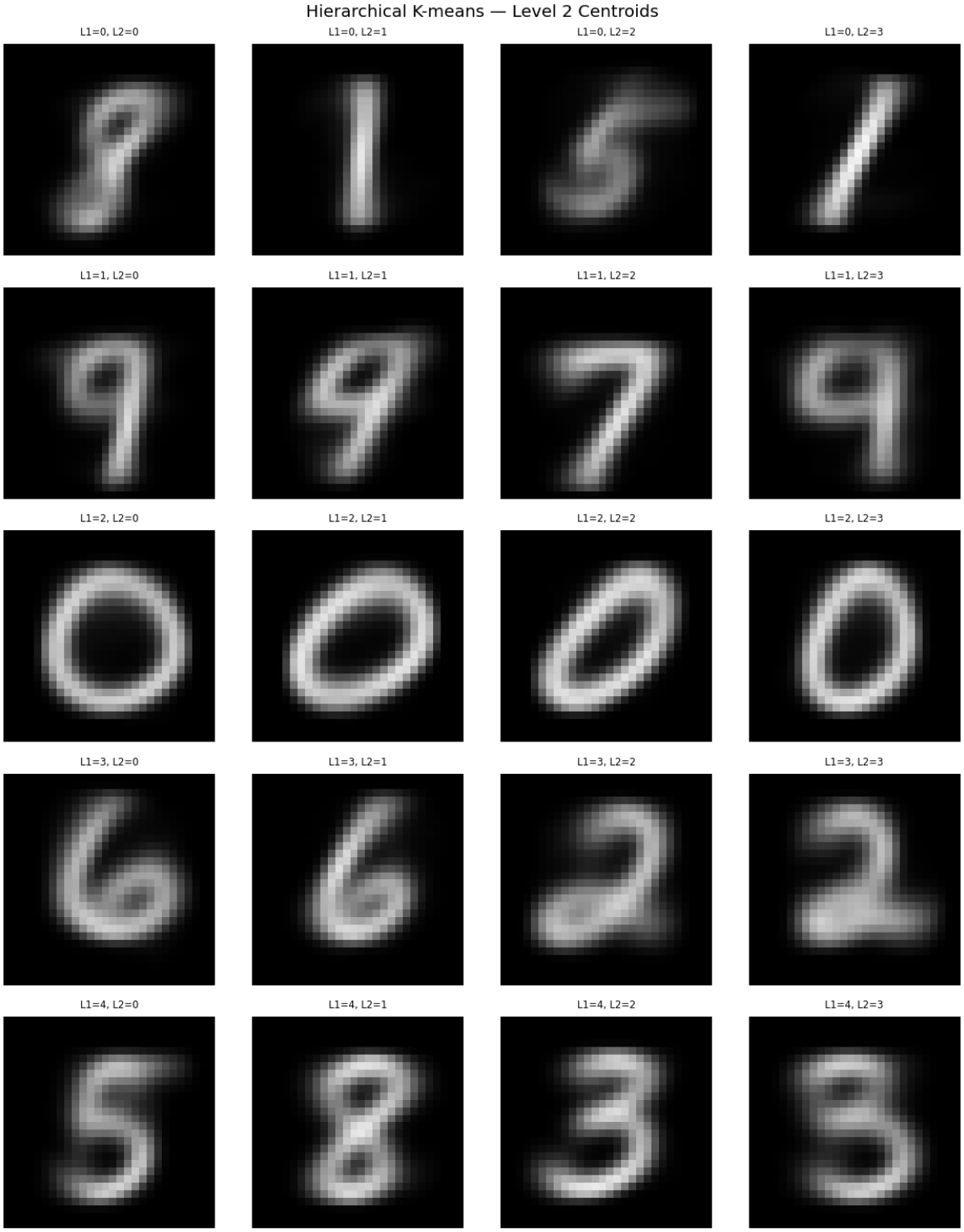


Figure 14: Level-2 centroids: each row is a coarse cluster, each column a sub-cluster within it.

Interpretability. Level-1 clusters correspond to broad visual categories (e.g., round shapes, vertical strokes, complex multi-curve digits). Level-2 clusters within each coarse group capture style

variation: thick vs. thin strokes, slant direction, loop closure, etc.

6.3 Digit Distribution in Level-1 Clusters

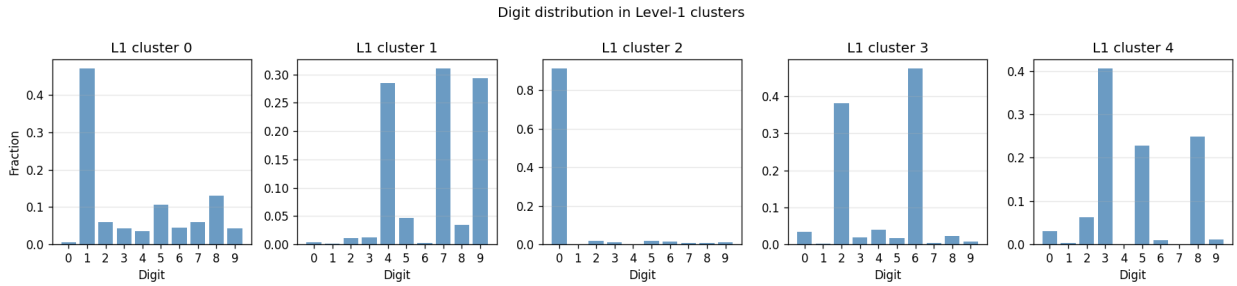


Figure 15: Digit distribution inside each level-1 cluster.

6.4 Flat vs. Hierarchical Comparison

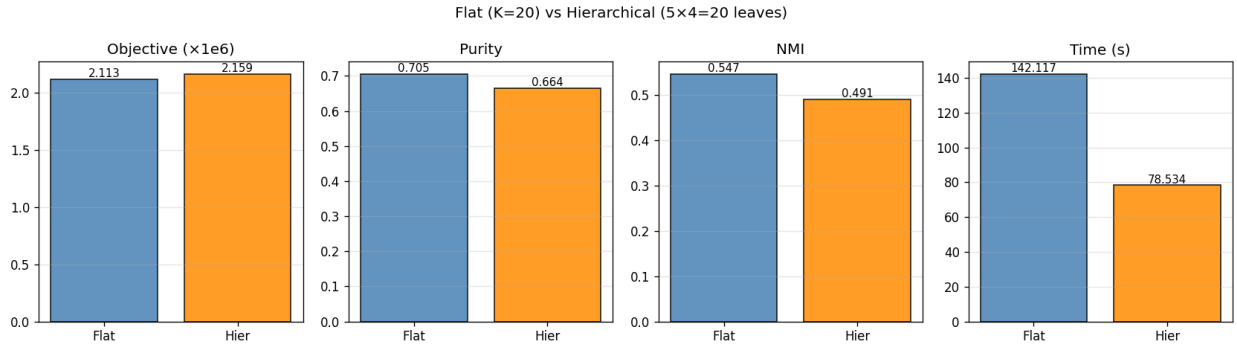


Figure 16: Flat K-means (K=20) vs. hierarchical K-means (5×4=20 leaves).

Table 2: Flat vs. hierarchical K-means at 20 leaf clusters.

Method	Leaves	Objective	Purity	Time (s)
Flat K-means (K=20)	20	2.113×10^6	0.705	142
Hierarchical (5×4)	20	2.159×10^6	0.664	**78**
Flat K-means (K=30)	30	1.986×10^6	0.758	∼260
Hierarchical (10×3)	30	2.038×10^6	0.681	∼58

Quantitative comparison.

Discussion.

- **Speed.** Hierarchical K-means is ≈45% faster at 20 leaves (78 s vs 142 s) and ≈4× faster at 30 leaves (∼58 s vs ∼260 s). The reason: each level-2 sub-problem runs on 5 000–17 000 points with K_2=3–4, not 60 000 points with K=20–30. The $\mathcal{O}(NKd)$ cost scales with the local cluster size.

- **Accuracy.** Flat K-means achieves lower objective (≈2% better) and higher purity (≈6% better) because it optimises the global criterion. Hierarchical K-means fixes level-1 centroids before level-2 runs; greedy errors at level 1 cannot be corrected.
- **Interpretability.** The tree structure provides a natural two-scale description. Level-1 clusters correspond to broad visual categories (round shapes, vertical strokes, etc.); level-2 clusters separate writing styles within each category.
- **Limitation.** Errors at level 1 propagate: a digit misassigned at the coarse level stays in the wrong branch through all subsequent splits. Without a global re-optimisation step, hierarchical greedy solutions cannot recover from early mistakes.

7 Task 5 — K-Medoids with Edit Distance

7.1 Representation

We converted each 28×28 image to a binary sequence using a pixel threshold of 0.3, then applied run-length encoding (RLE) row by row. The RLE sequence records alternating run lengths of background (0) and ink (1) pixels; typical lengths are 20–60 for MNIST digits.

Figure 17 shows the original image, the binary threshold, the RLE histogram, and the flattened binary sequence for three example images.

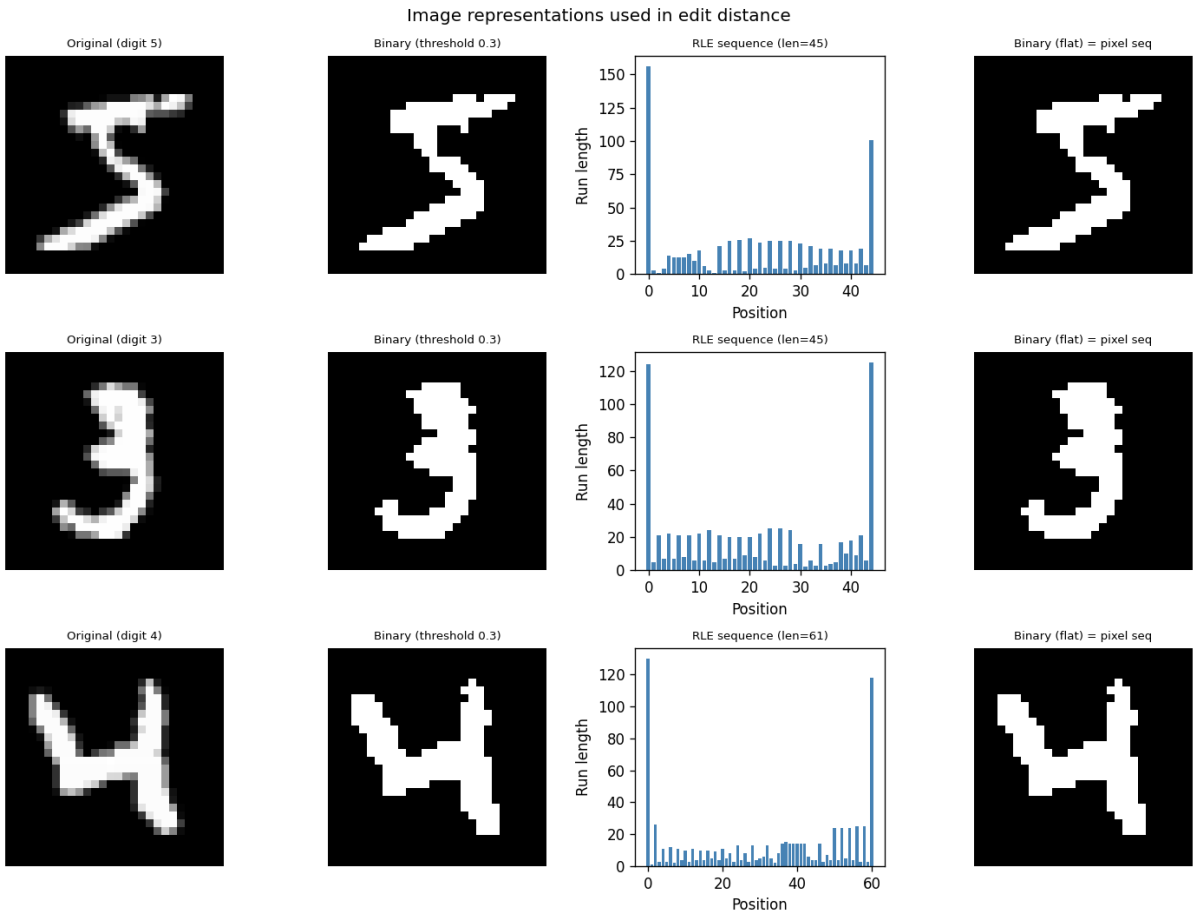


Figure 17: Image representations for the edit-distance experiment. Columns: original, binary, RLE bar chart, binary sequence.

7.2 Edit Distance

We used the Levenshtein edit distance with unit insertion, deletion, and substitution costs, computed via standard DP. Pairwise distances for 500 images were pre-computed in $\mathcal{O}(N^2L^2)$ time, where L is the typical sequence length.

7.3 Why the Standard K-Means Update Breaks

The standard K-means centroid update is

$$\mu_k^{(t+1)} = \frac{1}{|C_k|} \sum_{i \in C_k} x_i.$$

This is only meaningful when points are vectors in a Euclidean space: the average is well-defined, is in the same space as the data, and minimises the within-cluster SSE.

For sequences with edit distance, the "average" of two sequences is not well-defined. Even if it were, the result might not be a valid sequence. The correct analogue is the **Fréchet mean**, which requires solving a combinatorial problem (sequence alignment) over all cluster members.

We instead use **K-medoids**: the representative of each cluster is the actual data point minimising the total edit distance to all other members.

7.4 K-Medoids Algorithm

1. Initialise K medoids by random selection.
2. **Assignment:** assign each point to its nearest medoid under edit distance.
3. **Update:** for each cluster, select the point minimising the total intra-cluster distance as the new medoid.
4. Repeat until medoids stabilise.

7.5 Results

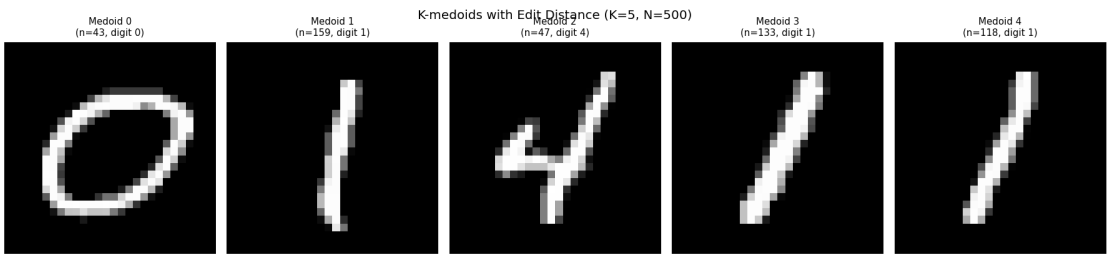


Figure 18: K-medoids prototypes (actual training images) for K=5, N=500.

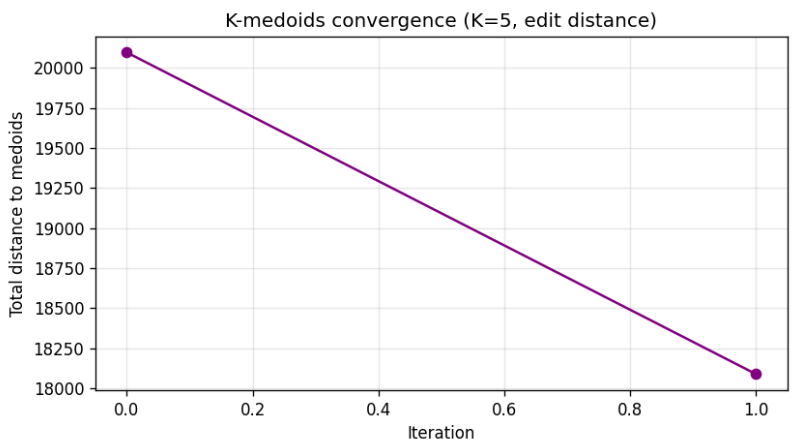


Figure 19: K-medoids convergence: total distance to medoids per iteration.

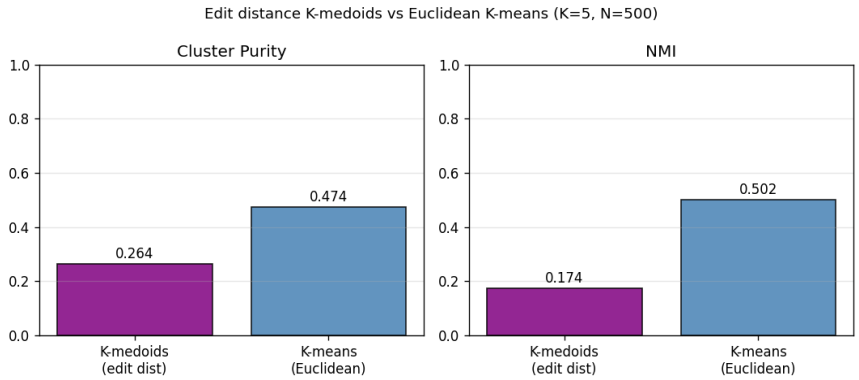


Figure 20: Comparison of edit-distance K-medoids and Euclidean K-means on the same 500-image subset (K=5).

Table 3: Edit-distance K-medoids vs. Euclidean K-means on 500 MNIST images (K=5).

Method	Purity	NMI	Distance pre-comp.
K-medoids (edit distance)	0.264	0.174	92.8 s
K-means (Euclidean)	0.474	0.502	<1 s

Discussion.

- **Computational cost.** Pre-computing the 500×500 RLE edit distance matrix takes 93 s. Scaling to 60 000 images would require $\mathcal{O}(60000^2)$ evaluations, taking roughly $93 \times (60000/500)^2 \approx 10^6$ s ($\sim$10 days) — totally prohibitive.
- **Quality.** Edit-distance K-medoids achieves substantially lower purity (0.264) and NMI (0.174) than Euclidean K-means (0.474/0.502) on the same 500 images. The RLE edit distance is a poor proxy for visual digit similarity: a small vertical shift of the digit changes run lengths dramatically even though the digit class is the same.
- **Sensitivity to representation.** The RLE encoding collapses spatial structure into a 1D sequence. Two images of the same digit written at slightly different vertical positions can have very different RLE sequences, leading to large edit distances. More sophisticated representations (e.g., chain-code contours or structured patch descriptors) would improve the distance metric but at the cost of additional preprocessing.
- **Key insight.** Leaving the Euclidean setting removes two convenient properties: (1) the centroid update has a closed-form solution (the arithmetic mean), and (2) the objective decomposes cleanly across clusters. K-medoids preserves the cluster decomposition but increases the per-step cost from $\mathcal{O}(NKd)$ (K-means) to $\mathcal{O}(N^2)$ (medoid finding), making it impractical at scale without approximations such as PAM or CLARA.

8 Summary and Conclusions

Table 4: Summary of clustering results across all tasks (K=10 or equivalent).

Method	Purity	NMI	Notes
Pixel K-means (K=10)	0.590	0.489	Baseline
DFT K-means, flat weights	0.394	0.304	Worse than pixel (phase lost)
DFT K-means, low-pass (σ=4)	0.431	0.359	Best DFT variant (val set)
Hierarchical K-means (5×4=20)	0.664	0.491	45% faster, −6% purity
K-medoids edit dist (K=5, N=500)	0.264	0.174	RLE repr.; 93 s for 500 pairs

Key findings.

1. **Initialisation matters.** K-means++ consistently produces lower final objectives than random initialisation, especially at large K. Practical rule: use 3–5 restarts with K-means++.

2. K=10 **is natural but not always optimal.** The elbow and silhouette suggest K=10–15. Larger K increases purity (finer-grained clusters) at the cost of interpretability; NMI plateaus around K=20.

3. **DFT representation has mild benefits.** Log-magnitude DFT features are slightly more robust than raw pixels under train/test splits. Low-pass weighting is most useful when the task is to capture global digit shape rather than writer style.

4. **Clusters generalise well.** Train and test metrics are nearly identical across all split ratios (50/50 to 90/10): pixel purity varies only ±0.025 across ratios. Pixel clusters show higher variance across different random splits (std = 0.035) but higher mean quality; DFT clusters are very stable (std = 0.003) but consistently lower quality.

5. **Hierarchical K-means trades optimality for speed and structure.** At 20 leaves: 45% faster (78 s vs 142 s), −6% purity (0.664 vs 0.705). The speed advantage grows with the number of leaves. The tree structure adds interpretability at the cost of greedy sub-optimality.

6. **Edit distance breaks the K-means framework.** Without a well-defined notion of mean, the centroid update cannot be applied. K-medoids is the natural replacement, but its $\mathcal{O}(N^2)$ distance-computation cost is prohibitive at scale (93 s for 500 images; ∼10 days for 60 000). Quality is also lower (purity = 0.26 vs 0.47 for Euclidean K-means) because the RLE edit distance is a poor proxy for visual digit similarity.